

# P2183R0

## Executors Review: Properties

Published Proposal, 2020-07-14

**This version:**

<http://wg21.link/p2183r0>

**Authors:**

[David Olsen](#)

[Ruslan Arutyunyan](#)

[Michael J. Voss](#)

[Michał Dominiak](#)

[Chris Kohlhoff](#)

[D.S. Hollman](#)

[Kirk Shoop](#)

[Inbal Levi](#)

**Audience:**

LEWG

**Project:**

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Table of Contents

### 1 Scope

### 2 Introduction

### 3 Examples

#### 3.1 Using standard properties

##### 3.1.1 Writing a custom executor

##### 3.1.2 Using `any_executor`

#### 3.2 Using custom properties

##### 3.2.1 Modifying our custom executor

3.2.2 Using `any_executor` again

## 4 Small Issues

### 5 Issues

- 5.1 Should properties be usable with non-executors?
- 5.2 Generic blocking adapter is not implementable
- 5.3 Non-movable properties
- 5.4 Polymorphic executor wrappers and prefer-only properties
- 5.5 `any_executor`'s `FIND_CONVERTIBLE_PROPERTY`
- 5.6 `any_executor<P...>::target` causes unused RTTI overhead
- 5.7 `bulk_guarantee` specification should match execution policies
- 5.8 What does "established property" mean?
- 5.9 `allocator_t::value` should not be static
- 5.10 `context` property has no constraints

### References

Informative References

## § 1. Scope

We were asked by the chair of LEWG to review how the Executors proposal [\[P0443R13\]](#) makes use of Properties [\[P1393\]](#). We were to look at all uses of properties within P0443, not just the sections that define properties.

We did not review P1393, where the properties mechanism is defined. But we did spend a fair amount of time reading and understanding P1393, and in the process we noticed a couple of issues in that paper, and we have offered suggested fixes.

The review team varied some during the effort, but ended up consisting of David Olsen, Ruslan Arutyunyan, Michael J. Voss, and Michał Dominiak, with contributions from the polymorphic executor review team led by Inbal Levi. Michael and Ruslan wrote all the examples. We were assisted in all our efforts by three of the many authors of P0443, Chris Kohlhoff, Daisy Hollman, and Kirk Shoop. The authors answered many questions, provided lots of background information, made useful suggestions, and were generally very helpful. In particular, Chris Kohlhoff's implementation of the properties mechanism and some other parts of executors that are part of [\[Asio\]](#) (see also [here](#) and [here](#)) made it much easier to write sample code and try out ideas.

## § 2. Introduction

Properties were originally suggested as a way to simplify the configuration and customization of the `execute` function for executors. At one time during the development of P0443, there were more than a dozen proposed execution functions or customization points, but even that large set covered only a subset of the different ways that execution could happen. The properties mechanism was developed to avoid this explosion in execution functions. Each characteristic of execution was separated into its own property, allowing for independent configuration of that characteristic. The properties mechanism (along with senders/receivers for configuration of returning results) reduced the number of execution functions down to two, `execute` and `bulk_execution`. The development of properties is explained in more detail in [\[P2033\]](#).

The properties mechanism was deemed to be useful beyond executors, so it was separated into its own paper, [\[P1393\]](#), and moved from namespace `std::execute` to namespace `std`. Properties are not used by the standard library outside of executors at the moment, but that is a possible future direction. A good introduction to properties in general, not specific to executors, is available in Chris's YouTube [video](#).

## § 3. Examples

We provide a set of examples that begin with a simple use of a standard property and standard executor. We then introduce a custom executor and show how support for a standard property can be added to that executor. We next implement a custom property and show how it can be used with a standard executor as well as our custom executor. Throughout the discussion we also provide example uses of `any_executor` that wrap both the standard and custom executor and use both standard and custom properties.

All of the examples were compiled against the [\[Asio\]](#) implementation of executors, so the code refers to many things in the `asio` namespace. Simply changing `asio` to `std` should result in standard-conforming code.

### § 3.1. Using standard properties

The example that follows shows a simple use of `blocking_t` with the executor returned from `static_thread_pool`. Our example has a race on the variable `i` if the function objects passed to `execute` execute concurrently -- this is intentional. We include this race purely to introduce an easily demonstrated effect of changing the blocking property; it is not expected that `blocking_t` will be used to avoid races in real applications.

```

void example_with_stp() {
    asio::static_thread_pool stp(8);

    // require is ok, since static_thread_pool supports blocking.always
    auto ex = asio::require(stp.executor(), asio::execution::blocking.always);
    std::cout << "Required blocking.always" << std::endl;

    // query is ok and informative, since static_thread_pool supports blocking
    if (asio::query(ex, asio::execution::blocking) == asio::execution::blocking.always)
        std::cout << "ex blocking.possibly" << std::endl;

    int i = 0;
    asio::execution::execute(ex, [&i]() { i = 1; });
    asio::execution::execute(ex, [&i]() { if (i == 1) i = 2; });
    std::cout << "Before wait i == " << i << std::endl;
    stp.wait();
    std::cout << "After wait i == " << i << std::endl;
}

```

Since the executor provided by `static_thread_pool` supports both `query` and `require` for `blocking_t`, we get a deterministic result, where `i==2` both before and after the wait on the `stp` context:

```

Required blocking.always
Before wait i == 2
After wait i == 2

```

### § 3.1.1. Writing a custom executor

It is relatively straightforward to add support for a standard property to a custom executor. Below we show a custom `toy_tbb_context` that has support for querying `blocking_t` and requiring `blocking_t::always_t`. Our toy context is built on top of the Threading Building Blocks (TBB) library which is an open-source C++ library for threading. The `toy_tbb_context` maintains a TBB `task_group`, which represents a group of tasks that the TBB library will schedule on to its internal thread pool. The `toy_tbb_context::executor_` has a member variable `blocking_value_` that holds the state of the property, which has a value of `blocking.possibly` by default. The function `query` returns the value and the function `require` returns a new `executor_` that has a reference to the same `toy_tbb_context` but with `blocking_value_` set to `blocking.always`. The

`executor_`'s member function `execute` calls `task_group::run` to implement a non-blocking `execute` and a calls `task_group::run_and_wait` to implement a blocking `execute`. The `toy_tbb_context` provides a member function `wait` that blocks until all tasks in its `task_group` are complete.

```

namespace toy {
    class toy_tbb_context {
    public:
        class executor_ {
        public:
            constexpr asio::execution::blocking_t query(asio::execution::blocking_t) const {
                return blocking_value_;
            }
            auto require(asio::execution::blocking_t::always_t val) const noexcept {
                return executor_{context_, val};
            }
            bool operator==(const executor_& other) const noexcept { return this == &other; }
            bool operator!=(const executor_& other) const noexcept { return this != &other; }
            executor_(const executor_& e) = default;
            ~executor_() = default;

            template<typename Invocable>
            void execute(Invocable &&f) const {
                if (blocking_value_ == asio::execution::blocking_t::always) {
                    context_.task_group_.run_and_wait(std::forward<Invocable>(f));
                } else {
                    context_.task_group_.run(std::forward<Invocable>(f));
                }
            }
        };

    private:
        friend toy_tbb_context;
        asio::execution::blocking_t blocking_value_;
        toy_tbb_context &context_;
        executor_(toy_tbb_context& context) noexcept : context_(context) {}
        executor_(toy_tbb_context& context,
                  const asio::execution::blocking_t& blocking_val) noexcept :
            context_(context), blocking_value_(blocking_val) {}
    };

    executor_ executor() { return executor_{*this}; }

    void wait() { task_group_.wait(); }

private:
    tbb::task_group task_group_;

```

```
};  
}
```

We can replace the `static_thread_pool` in our previous example with our `toy_tbb_context` and, due to its support of `blocking_t`, also get deterministic results, where `i == 2` both before and after the wait.

```
void example_with_toy_tbb() {  
    toy::toy_tbb_context ttc;  
  
    // require is ok, since toy_tbb_context supports blocking.always  
    auto ex = asio::require(ttc.executor(), asio::execution::blocking.always)  
  
    // query is ok and informative, since toy_tbb_context supports blocking_t  
    if (asio::query(ex, asio::execution::blocking) == asio::execution::blocki  
        std::cout << "ex blocking.possibly" << std::endl;  
  
    int i = 0;  
    asio::execution::execute(ex, [&i]() { i = 1; });  
    asio::execution::execute(ex, [&i]() { if (i == 1) i = 2; });  
    std::cout << "Before wait i == " << i << std::endl;  
    ttc.wait();  
    std::cout << "After wait i == " << i << std::endl;  
}
```

It should be noted that executors, senders and schedulers are **not** required to support all of the standard properties. We implemented a `toy_tbb_context_no_support` as follows without any support for properties. In this implementation, the `execute` function always calls the non-blocking `task_group::run`.

```

namespace toy {
    class toy_tbb_context_no_support {
    public:
        class executor_ {
        public:
            bool operator==(const executor_& other) const noexcept { return this
            bool operator!=(const executor_& other) const noexcept { return this
            executor_(const executor_& e) = default;
            ~executor_() = default;

            template<typename Invocable>
            void execute(Invocable &&f) const {
                context_.task_group_.run(std::forward<Invocable>(f));
            }

        private:
            toy_tbb_context_no_support &context_;
            friend toy_tbb_context_no_support;
            executor_(toy_tbb_context_no_support& context) noexcept : context_(co
        };

        executor_ executor() { return executor_{*this}; }

        void wait() {
            task_group_.wait();
        }

    private:
        tbb::task_group task_group_;
    };
}

```

We can no longer plug this context in directly for `static_thread_pool` in our example, because the use of `require` will not compile. However if we are unsure that an executor will provide `require` for a requirable property, we can use `prefer` instead, which will call `require` only if it is supported; otherwise, it returns an executor that has the same properties established as the executor that was passed to it. It is also intended that all standard properties have reasonable defaults, so `query` will return an accurate but broad result even if the executor does not explicitly support `query`. In the case of `blocking_t`, the default `query` returns `blocking::possibly`, which is never incorrect. Therefore to use the `toy_tbb_context_no_support`, we update our example as shown:



```

void example_with_toy_tbb_no_support() {
    toy::toy_tbb_context_no_support ttc;

    // prefer is ok, even for an executor that does not explicitly support blocking
    // it returns an executor with the same properties
    auto ex = asio::prefer(ttc.executor(), asio::execution::blocking.always);
    std::cout << "Preferred blocking.always" << std::endl;

    // query is ok, even for an executor that does not explicitly support blocking
    // the result is broad but still useful
    if (asio::query(ex, asio::execution::blocking) == asio::execution::blocking.possibly)
        std::cout << "ex blocking.possibly" << std::endl;

    std::atomic<int> i = 0;
    asio::execution::execute(ex, [&i]() { i = 1; });
    asio::execution::execute(ex, [&i]() { int j = 1; i.compare_exchange_strong(0, 1); });
    std::cout << "Before wait i == " << i << std::endl;
    ttc.wait();
    std::cout << "After wait i == " << i << std::endl;
}

```

Since, without blocking, our example now contains a real race, we use an `atomic<int>` and `compare_exchange_strong`. Depending on which operation on `i` occurs first, the result after the wait may be 1 or 2. An example output, where the second call to `execute` completes first, and therefore `i == 1`, is shown below:

```

Preferred blocking.always
ex blocking.possibly
Before wait i == 0
After wait i == 1

```

### § 3.1.2. Using `any_executor`

We can wrap the executor types returned by `static_thread_pool`, `toy_tbb_context` and `toy_tbb_context_no_support` in the polymorphic wrapper `any_executor`. For demonstration, we introduce a trivial `algorithm` that contains the two calls to `execute` as show below. Because the executor types of both `static_thread_pool` and `toy_tbb_context` support `require` for `blocking_t::always_t`, they can be both be wrapped in `any_exec_type`.

```

using any_exec_type =
    asio::execution::any_executor<asio::execution::blocking_t,
        asio::execution::blocking_t::always_t>;

void algorithm(any_exec_type ex0) {
    // since blocking_t::always_t is supported, we know we can require it
    auto ex = asio::require(ex0, asio::execution::blocking.always);
    std::cout << "Required blocking.always" << std::endl;
    int i = 0;
    asio::execution::execute(ex, [&i]() { i = 1; });
    // Since it's blocking, we don't need the compare_exchange
    asio::execution::execute(ex, [&i]() { if (i == 1) { i = 2; } });
    std::cout << "i == " << i << std::endl;
}

void example_with_stp() {
    asio::static_thread_pool stp(8);
    algorithm(stp.executor());
    stp.wait();
}

void example_with_toy_tbb() {
    toy::toy_tbb_context ttc;
    algorithm(ttc.executor());
    ttc.wait();
}

```

Our `toy_tbb_context_no_support` does not, however, support `require` for `blocking_t::always_t` and so it cannot be wrapped with `any_exec_type`. If we are unsure if an executor supports `require` for a property, we can again rely on `prefer` and create a different `any_executor` type that uses the `prefer_only` property adapter.



```

namespace toy {
    struct tracing_t
    {
        tracing_t() = default;
        constexpr tracing_t(bool val) : value_(val) {}

        template <typename T>
        static constexpr bool is_applicable_property_v = asio::execution::execu
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = true;
        using polymorphic_query_result_type = bool;

        constexpr explicit operator bool() const
        {
            return value_;
        }
    private:
        bool value_{false};
    };

    inline constexpr tracing_t tracing;

    template<typename E>
    constexpr bool query(const E&, const tracing_t&) { return false; }
}

```

### § 3.2.1. Modifying our custom executor

The support in our custom `toy_tbb_executor` is similar to the support for `blocking_t`. We show only the additions and changes below:

```

namespace toy {
    class toy_tbb_context {
    public:
        class executor_ {
        public:
            // ...
            constexpr bool query(const tracing_t&) const noexcept {
                return static_cast<bool>(tracing_value_);
            }
            auto require(const tracing_t& val) const noexcept {
                return executor_{context_, blocking_value_, val};
            }
            // ...
            template<typename Invocable>
            void execute(Invocable &&f) const {
                if (tracing_value_)
                    ++context_.executions_;

                if (blocking_value_ == asio::execution::blocking_t::always) {
                    context_.task_group_.run_and_wait(std::forward<Invocable>(f));
                } else {
                    context_.task_group_.run(std::forward<Invocable>(f));
                }
            }
        private:
            // ...
            tracing_t tracing_value_;
            // ...
            executor_(toy_tbb_context& context,
                    const asio::execution::blocking_t& blocking_val,
                    const tracing_t& tracing_val) noexcept :
                context_(context),
                blocking_value_(blocking_val),
                tracing_value_(tracing_val) {}
        };
        // ...
        int traced_executions() const noexcept { return executions_; }
    private:
        // ...
        std::atomic<int> executions_;
    };
}

```

We can now `require` this property to turn tracing on and off for the `toy_tbb_context::executor_` as shown below:

```
void example_with_toy_tbb() {
    toy::toy_tbb_context ttc;

    // require is ok, since toy_tbb_context supports blocking.always
    auto ex = asio::require(ttc.executor(), asio::execution::blocking.always);
    std::cout << "Required blocking.always" << std::endl;

    // query is ok and informative, since toy_tbb_context supports blocking_t
    if (asio::query(ex, asio::execution::blocking) == asio::execution::blocking_t::always)
        std::cout << "ex blocking.possibly" << std::endl;

    int i = 0;
    asio::execution::execute(ex, [&i]() { i = 1; });
    asio::execution::execute(ex, [&i]() { if (i == 1) i = 2; });
    ttc.wait();
    std::cout << "i == " << i << std::endl;
    std::cout << "traced_executions == " << ttc.traced_executions() << std::endl;

    i = 0;
    auto tex = asio::require(ex, toy::tracing_t{true});
    std::cout << "Required toy::tracing_t{true}" << std::endl;
    asio::execution::execute(tex, [&i]() { i = 1; });
    asio::execution::execute(tex, [&i]() { if (i == 1) i = 2; });
    ttc.wait();
    std::cout << "i == " << i << std::endl;
    std::cout << "traced_executions == " << ttc.traced_executions() << std::endl;
}
```

The output of the above code is as follows:

```
Required blocking.always
i == 2
traced_executions == 0
Required toy::tracing_t{true}
i == 2
traced_executions == 2
```

`static_thread_pool` does not support this property and since it is a standard executor type, we cannot add support. However, we have provided a reasonable default for `query`, tracing is off. So, we can both `query` and `prefer` this property on the executor type from `static_thread_pool`, with `prefer` returning an executor with the same properties as the executor that was passed to it.

```
void example_with_stp() {
    asio::static_thread_pool stp(8);

    // require is ok, since static_thread_pool supports blocking.always
    auto ex = asio::require(stp.executor(), asio::execution::blocking.always);
    std::cout << "Required blocking.always" << std::endl;

    // query is ok and informative, since static_thread_pool supports blocking
    if (asio::query(ex, asio::execution::blocking) == asio::execution::blocking)
        std::cout << "ex blocking.possibly" << std::endl;

    int i = 0;
    // prefer is ok, even for an executor that does not explicitly support tracing
    // it returns an executor with the same properties
    auto tex = asio::prefer(ex, toy::tracing_t{true});
    std::cout << "Preferred toy::tracing_t{true}" << std::endl;
    asio::execution::execute(tex, [&i]() { i = 1; });
    asio::execution::execute(tex, [&i]() { if (i == 1) i = 2; });
    stp.wait();
    std::cout << "i == " << i << std::endl;
}
```

### § 3.2.2. Using `any_executor` again

Lastly, we can wrap both `static_thread_pool` and `toy_tbb_context` in `any_executor` and use both the standard `blocking_t` and the custom `toy::tracing_t` properties. Since `static_thread_pool` does not explicitly support `toy::tracing_t`, we must again use the `prefer_only` adapter if we want our algorithms to support both executors. In our example, we also demonstrate that implicit conversions support construction of an `any_executor` from another `any_executor` that has the same supported properties even if they are in a different order in the template argument list.

```

using any_exec_type =
    asio::execution::any_executor<asio::execution::blocking_t,
                                  asio::execution::prefer_only<toy::tracing_t,
                                  asio::execution::blocking_t::always_t>;

using any_almost_same_exec_type =
    asio::execution::any_executor<asio::execution::blocking_t,
                                  asio::execution::blocking_t::always_t,
                                  asio::execution::prefer_only<toy::tracing_t,
                                  asio::execution::blocking_t::always_t>;

void algorithm(any_exec_type ex) {
    auto tt_ex = asio::require(ex, asio::execution::blocking.always);
    std::cout << "Required blocking.always" << std::endl;
    if (asio::query(tt_ex, toy::tracing))
        std::cout << "Using tracing" << std::endl;

    int i = 0;
    asio::execution::execute(tt_ex, [&i]() { i = 1; });
    asio::execution::execute(tt_ex, [&i]() { if (i == 1) i = 2; });
    std::cout << "i == " << i << std::endl;
}

void algorithm_tracing(any_almost_same_exec_type ex)
{
    auto tracing_exec = asio::prefer(ex, toy::tracing_t{true});
    algorithm(tracing_exec);
}

void combined_example() {
    toy::toy_tbb_context ttc;
    algorithm(ttc.executor());
    std::cout << "traced_executions == " << ttc.traced_executions() << std::endl;
    algorithm_tracing(ttc.executor());
    std::cout << "traced_executions == " << ttc.traced_executions() << std::endl;

    asio::static_thread_pool stp(4);
    algorithm(stp.executor());
    std::cout << "traced_executions == " << ttc.traced_executions() << std::endl;
    algorithm_tracing(stp.executor());
    std::cout << "traced_executions == " << ttc.traced_executions() << std::endl;
}

```



When run, this example turns on tracing for the `toy_tbb_context` executor when `algorithm_tracing` is called:

```
Required blocking.always
i == 2
traced_executions == 0
Required blocking.always
Using tracing
i == 2
traced_executions == 2
Required blocking.always
i == 2
traced_executions == 2
Required blocking.always
i == 2
traced_executions == 2
```

## § 4. Small Issues

For small issues that have an obvious resolution, mostly wording bugs, we created issues in the [Executors GitHub](#) repository. The GitHub issues are listed here, but they can be resolved easily by the authors of P0443 and don't need to be discussed in LEWG.

- any\_executor section is in the wrong place [#488](#)
- Use of undefined name "Property" in behavioral properties [#489](#)
- Wrong property type in static\_thread\_pool sender and executor summaries [#490](#)
- Incorrect syntax in description of allocator\_t<ProtoAllocator> property [#491](#)
- bad\_executor refers to non-existent any\_executor::bulk\_execute [#493](#)
- S::Ei should be S::Ni [#494](#)
- prefer\_only example is out of date [#498](#)

## § 5. Issues

Here is a list of issue that we think deserve more discussion in LEWG. Some of the issues have a suggested resolution which we would like LEWG to approve and the P0443 authors to implement. Some issues merely identify a problem that we would like the authors of P0443 to solve, possibly with

guidance from LEWG. Any changes made in response to [§ 5.2 Generic blocking adapter is not implementable](#) or [§ 5.7 bulk\\_guarantee specification should match execution policies](#) should also be reviewed by SG1.

## § 5.1. Should properties be usable with non-executors?

All of the properties defined in P0443 have

```
template <class T>
    static constexpr bool is_applicable_property_v = executor<T>;
```

This limits the property to types that satisfy the `executor` concept. Any attempt to apply the property to something that is not an `executor` is normally ill-formed, even if that something has the necessary `query`, `require`, or `prefer` specializations that would otherwise make everything work. For example, the first bullet in the specification of `std::prefer` in P1393 is:

- If `is_applicable_property_v<T, Prop0> && Prop0::is_preferable` is not a well-formed constant expression with value `true`, `std::prefer(E, P0, Pn...)` is ill-formed.

If the `is_applicable_property_v` check fails, the call to `std::prefer` is ill-formed, rather than proceeding to the fallback of returning `E` unchanged that is normally used when a property can't be applied to a particular executor.

The specification for the `static_thread_pool` sender types lists member functions that support the properties `context`, `blocking`, `relationship`, `outstanding_work`, `allocator`, `bulk_guarantee`, and `mapping`. The specification for the `static_thread_pool` scheduler types lists member functions that support the properties `context` and `allocator`.

Some senders satisfy the `executor` concept (because `std::execution::execute` has a fallback that accepts a sender as the first argument). It seems that `static_thread_pool`'s senders satisfy `executor` (because they are noexcept-copyable and equality-comparable), so the property should work with `static_thread_pool`'s senders. But there is no guarantee that other senders satisfy `executor`. `static_thread_pool`'s scheduler types are not guaranteed to satisfy `executor`, so the properties likely cannot be applied to them.

It seems that it would be useful to be able to apply these properties to senders. Senders have access to an executor under the covers. When users get the sender from `schedule` and then pass it to `submit`

or `connect` or to an asynchronous algorithm, they never have direct access to the executor. Users who want to customize or tune the executor in some way have to go through the sender to get to the executor, but there is no standard way to do that.

Similar reasoning applies to schedulers. Schedulers dole out senders which are associated with executors. It would be helpful for users to tell the scheduler what properties they want for their executors, and the scheduler would only create senders that are associated with executors with those properties.

One possible solution to this discrepancy is to change the `is_applicable_property_v` member of each executor property from `executor<T>` to `executor<T> || sender<T> || scheduler<T>`. That would allow all senders and schedulers to support these properties if desired. Support would still be opt-in; senders and schedulers would have to define the necessary overloads of `query`, `require`, and/or `prefer` to actually support the properties.

The other way for senders and schedulers to support these properties is to specialize `std::is_applicable_property<T, P>` as described in "Property applicability trait" in [P1393]: "It may be specialized to indicate applicability of a property to a type." Doing it this way can be verbose, requiring a specialization for each supported property by each non-executor class template.

The inconsistency in the use of properties by `static_thread_pool`'s schedulers needs to be dealt with in some way. The three obvious choices are:

1. Remove the support for properties, deleting the `query` and `require` functions from the scheduler types specification in section 2.5.3. This is not an ideal solution because it removed functionality that we feel is useful.
2. Change the `is_applicable_property_v` members of all the relevant properties to have the value `executor<T> || sender<T> || scheduler<T>`. There are concerns that this disjunction, while not part of a concept definition, could still increase compile-time unnecessarily.
3. Change the `is_applicable_property_v` members of all the relevant properties to have the value `true`. Some members of our group think that using a property defined in the `execution` namespace is enough of an opt-in, and don't think that properties need to restrict their applicability.
4. Specify that `std::is_applicable_property<T, P>` is specialized to have a base characteristic of `true_type` for scheduler types and the relevant properties. This is the most cumbersome solution, but it is the mechanism endorsed by P1393.

The review group agreed that this is an issue that needs to be addressed, but could not agree on the best way to address it. There was some support for all options other than option #1. In the current [Asio] implementation of `static_thread_pool`, a different approach was taken to solving this

issue: the same template class implements the thread pool's executors, senders, and schedulers. So that class satisfies all three `executor`, `sender`, and `scheduler` concepts.

Even though properties should already work as specified for `static_thread_pool`'s sender types, it would be good if a similar approach was taken with the sender types. Or at least add a non-normative note pointing out that the properties work because the sender types satisfy the `executor` concept.

## § 5.2. Generic blocking adapter is not implementable

The specification for `blocking_adaptation_t::allowed_t` in section 2.2.12.2.1 contains:

This customization uses an adapter to implement the `blocking_adaptation_t::allowed_t` property.

```
template<class Executor>
    friend see-below require(Executor ex, blocking_adaptation_t::allowed_t)
```

This allows the `blocking_adaptation.allowed` property to be applied to any executor. If the executor does not support the `blocking_adaptation.allowed` property directly, then this `require` function will create a wrapper around the executor with the `blocking_adaptation.allowed` property established.

The specification for `blocking_t::always_t` in section 2.2.12.1.1 contains:

If the executor has the `blocking_adaptation_t::allowed_t` property, this customization uses an adapter to implement the `blocking_t::always_t` property.

```
template<class Executor>
    friend see-below require(Executor ex, blocking_t::always_t);
```

This allows the `blocking.always` property to be applied to any executor that has the `blocking_adaptation.allowed` property. If the executor does not support the `blocking.allowed` property directly, then this `require` function will create a wrapper around the executor where calls to `execute` and `bulk_execute` will block.

The way these two properties are specified, it is possible to turn any executor into a blocking executor:

```
// This should work with almost any executor
executor auto blocking_ex =
    std::require(
        std::require(ex, blocking_adaptation.allowed),
        blocking.always);
static_assert(std::query(blocking_ex, blocking) == blocking.always);
```

(It is possible for executor types to prevent these properties from being applied, but it takes extra work on the part of the executor. If executor type `Ex` wants to block the `blocking_adaptation.allowed` property, it would have to specialize `std::is_applicable_property<Ex, blocking_adaptation_t::allowed_t>` to be false, or it would have to define a deleted non-member function `void require(Ex, blocking_adaptation_t::allowed_t) = delete;` to be a better match during overload resolution than the `require` function defined by `blocking_adaptation_t::allowed_t`. For executor types that don't go through this extra effort to block either of these two properties, the double-require example above will compile successfully.)

This is a problem because it is not possible to implement a blocking wrapper around every executor. Without understanding the details of the wrapped executor's execution context, the wrapper can't choose a synchronization primitive that will be known to work between the current thread and the wrapped executor's execution context. For example, using a mutex or a condition variable for synchronization won't work if the execution context is a fiber or certain GPUs. The standard properties for executors don't provide enough information about the execution context for the wrapper to know the proper way to block.

A standard construct that makes it easy for users to write code that won't work is poor design. Changes need to be made to the `blocking_adaptation` property. Here are some options. Some of them can be combined together.

1. **Remove `blocking_adaptation` property; remove the generic `blocking.always` adapter.**

Each executor knows best how to make it a blocking executor. It is not possible to have a general purpose blocking adapter that always works correctly. Converting a non-blocking executor to a blocking executor should be done by the executor itself, by implementing `std::require(Ex, blocking_t::always_t)`. (If this approach is taken, the `blocking.always` property should become preferable. `blocking_t::always_t::is_preferable` should be changed from `false` to `true`.)

2. **Remove the generic `blocking_adaptation.allowed` adapter.** The

`blocking_adaptation` property would still exist, as would the generic `blocking.always` adapter. But there would be no generic adapter for adding the

`blocking_adaptation.allowed` property to an arbitrary executor. Executors where the standard `blocking.allowed` adapter will work correctly would come with the `blocking_adaptation.allowed` property already established. More unusual executors that won't work with the standard `blocking.always` adapter won't support the `blocking_adaptation.allowed` property at all, and they will either provide their own implementation of `std::require(Ex, blocking_t::always_t)` or not provide any means to become a blocking executor.

3. **The `blocking.always` adapter requires a thread-based executor.** It is possible to write a generic blocking adapter for executors that operate on standard threads, because many of the standard synchronization primitives are known to work correctly in that situation. To implement this requirement for thread-based executors, change the prerequisite for the blocking adapter to, "If the executor has the `blocking_adaptation_t::allowed_t` property and has either the `mapping_t::thread_t` or the `mapping_t::new_thread_t` property,".
4. **Change the name of `blocking_adaptation` and/or `blocking` to include "thread".** Blocking adapters work best with thread-based executors. This could be better communicated to users by include "thread" in the property names, such as `thread_blocking_adaptation` or `thread::blocking_adaptation`.

We recommend option #1, which had the support of the majority of the review group.

If the committee feels strongly that a generic adapter for `blocking.always` should exist, then we recommend both option #2 and option #3.

## § 5.3. Non-movable properties

[P1393] does not require properties to be movable or copyable. This was an intentional decision, and not an oversight. While all the properties defined in P0443 are movable and copyable, user-defined executors may define custom properties that are not movable or copyable. The executors framework, such as `any_executor`, should work correctly with such properties. That means any place that a generic property is passed as a parameter, it must be passed by `const&` and not by value. We have identified these places that should be changed to handle non-movable properties:

In `any_executor`, the functions `query`, `require`, and `prefer` accept a generic property by value. They need to be changed to accept it by `const&`. Using `require` as an example:

```
template <class Property>
any_executor require(const Property& p) const;
```

(We have heard that the `any_executor` review group is recommending changing these functions to take the property argument by forwarding reference. We are fine with that, and don't believe that it would interfere with solving the non-movable property problem.)

`any_executor`'s exposition-only `FIND_CONVERTIBLE_PROPERTY(p, pn)` operation uses `std::is_convertible` to find the appropriate property. This operation won't find non-movable properties because a non-movable type cannot be converted to itself. The definition should be changed as follows:

In several places in this section the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is used. All such uses mean the first type `P` in the parameter pack `pn` for which `std::is_same_v<p, P> is true or std::is_convertible_v<p, P> is true`. If no such type `P` exists, the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is ill-formed.

Chris Kohlhoff has tested this and has found these changes to be necessary and sufficient to support non-movable properties in `any_executor`.

Changing `prefer_only` to work with non-movable properties will be harder. The specification for `prefer_only` lists a public data member of the wrapper property type:

```
template<class InnerProperty>
struct prefer_only
{
    InnerProperty property;
```

The data member `property` should be an exposition-only member, not part of the public interface of the class. (If the public data member is intentional, then `prefer_only` cannot support non-copyable properties.) The `prefer_only` constructor should state that it keeps a copy of the wrapped property if `InnerProperty` is copyable, and that it keeps a reference to the wrapped property otherwise.

Even though end users should never have to create `prefer_only` objects (the end user only sees `prefer_only` in the type list for `any_executor`), the implementation of `any_executor` likely needs to create temporary `prefer_only` objects. (Asio's implementation of `any_executor` creates such temporaries.) Therefore, `prefer_only`'s ability to support non-movable properties affects the usability of `any_executor`.

## § 5.4. Polymorphic executor wrappers and prefer-only properties

The specification of customization point `std::prefer` in [P1393] only checks for functions named `require`. If requiring the property fails, then `std::prefer` returns the original executor unchanged. The customization point never looks for functions named `prefer`.

(`std::prefer` is supposed to return the original object unchanged if requiring the property fails. But a wording bug in P1393 leaves out that fallback action. There needs to be an extra bullet inserted between the fourth and fifth bullets, so it will be third from the end:

- `Otherwise, E if N == 0.`

This change must be made, even if the other proposed changes in this section are rejected.)

This specification of `std::prefer` makes it difficult for `any_executor`, or any other generic code that forwards `require` and `prefer` operations on properties, to correctly handle a property that can be preferred but not required.

As an example, assume that some property `prefer_nice_t` defines `is_requirable = false` and `is_preferable = true`. Assume that executor type `maybe_nice_exec` supports the property `prefer_nice_t`, such that given:

```
maybe_nice_exec e = ...;
auto nice_e = std::prefer(e, prefer_nice);
```

the executor `nice_e` has the property `prefer_nice_t`.

Now assume that we wrap a `maybe_nice_exec` in an `any_executor` and try to do the same `prefer` call:

```
maybe_nice_exec e = ...;
any_executor<prefer_nice_t> wrapped{e};
auto nice_wrapped = std::prefer(wrapped, prefer_nice);
```

Given the current specification of `any_executor`, this code will not behave as expected. `nice_wrapped` will not have the `prefer_nice_t` property. The call to `std::prefer` will try to call `any_executor`'s `require` member function. The specification for that says:



*Returns:* A polymorphic wrapper whose target is the result of `std::require(e, p)`, where `e` is the target object of `*this`.

The call to `std::require(e, p)` will fail immediately because `is_requirable` is false. The call will not be passed on to `maybe_nice_exec` to be handled. So the original `std::prefer` call will return the `any_executor` unchanged rather than returning a new `any_executor` whose target executor has the `prefer_nice_t` property.

To get the correct result, `any_executor`'s `require` function has to call `std::prefer` instead of `std::require` when it was called by `std::prefer`. But it doesn't have any foolproof way to know that it was called by `std::prefer` rather than `std::require`.

When generic code such as `any_executor` is forwarding calls to `std::require` and `std::prefer` from one object to a different object, it is not possible to forward the calls correctly because the important information of whether the original call was `require` or `prefer` is lost part way through the process.

The review group recommends that this issue be fixed by changing the specification of `std::prefer` in [\[P1393\]](#), having it check for functions named `require` first, then for functions named `prefer`, then falling back to returning the object unchanged.

The name `prefer` denotes a customization point object. The expression `std::prefer(E, P0, Pn...)` for some subexpressions `E` and `P0`, and where `Pn...` represents `N` subexpressions (where `N` is 0 or more, and with types `T = decay_t<decltype(E)>` and `Prop0 = decay_t<decltype(P0)>`) is expression-equivalent to:

- If `is_applicable_property_v<T, Prop0>` && `Prop0::is_preferable` is not a well-formed constant expression with value `true`, `std::prefer(E, P0, Pn...)` is ill-formed.
- Otherwise, `E` if `N == 0` and the expression `Prop0::template static_query_v<T> == Prop0::value()` is a well-formed constant expression with value `true`.
- Otherwise, `(E).require(P0)` if `N == 0` and the expression `(E).require(P0)` is a valid expression.
- Otherwise, `require(E, P0)` if `N == 0` and the expression `require(E, P0)` is a valid expression with overload resolution performed in a context that does not include the declaration of the `require` customization point object.
- Otherwise, `(E).prefer(P0)` if `N == 0` and the expression `(E).prefer(P0)` is a valid expression.
- Otherwise, `prefer(E, P0)` if `N == 0` and the expression `prefer(E, P0)` is a valid expression with overload resolution performed in a context that does not include the declaration of the `prefer` customization point object.
- Otherwise, `E` if `N == 0`.
- Otherwise, `std::prefer(std::prefer(E, P0), Pn...)` if `N > 0` and the expression `std::prefer(std::prefer(E, P0), Pn...)` is a valid expression.
- Otherwise, `std::prefer(E, P0, Pn...)` is ill-formed.

Along with this change, `any_executor`'s `prefer` function should be changed from a non-member function to a member function. The specification of the `prefer` function doesn't need to otherwise change. (§ 5.3 Non-movable properties and § 5.5 `any_executor`'s `FIND_CONVERTIBLE_PROPERTY` suggest other changes to `any_executor`'s `prefer` function, but those are separate issues.)

Please note that only generic code that wants to correctly forward `std::require` and `std::prefer` calls will need to define a `prefer` function. The vast majority of executors or other property-aware classes will only need to define `require` for the properties that they support. That `require` function will do the right thing for `std::prefer` for all normal property-aware classes.

This is the preferred solution for [\[Asio\]](#), which has already implemented this change.

## § 5.5. `any_executor`'s `FIND_CONVERTIBLE_PROPERTY`

The type-erased executor wrapper `any_executor` defines an exposition-only expression `FIND_CONVERTIBLE_PROPERTY(p, pn)`:

In several places in this section the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is used. All such uses mean the first type `P` in the parameter pack `pn` for which `std::is_convertible_v<p, P>` is true. If no such type `P` exists, the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is ill-formed.

This exposition-only expression is used in the specification of `any_executor`'s `require` function:

```
template <class Property>
any_executor require(Property p) const;
```

*Remarks:* This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_requirable` is well-formed and has the value `true`.

*Returns:* A polymorphic wrapper whose target is the result of `std::require(e, p)`, where `e` is the target object of `*this`.

Consider this example:

```
any_executor<blocking_t, blocking_t::never_t> ex{my_executor};
ex.require(blocking.never);
```

The `ex.require(blocking.never)` call is ill-formed. `FIND_CONVERTIBLE_PROPERTY` finds `blocking_t`, because it is the first one in the list that is convertible from a `blocking_t::never_t`. But `blocking_t::is_requirable` is false, so this particular overload of `require` doesn't participate in overload resolution. (This is not a contrived example. This pattern is in the first `any_executor` [example](#) in this paper.)

The fix is to check for `is_convertible` and `is_requirable` at the same time, rather than checking `is_convertible` first and `is_requirable` later. The specification of `any_executor`'s `require` becomes (with the change from [§ 5.3 Non-movable properties](#) thrown in):

```
template <class Property>
any_executor require(const Property& p) const;
```

Let FIND\_REQUIRABLE\_PROPERTY(p, \_pn) be the first type P in the parameter pack pn for which

- is\_same\_v<p, P> is true or is\_convertible\_v<p, P> is true, and
- P::is\_requirable is true.

If no such P exists, the operation FIND\_REQUIRABLE\_PROPERTY(p, \_pn) is ill-formed.

*Remarks:* This function shall not participate in overload resolution unless

~~FIND\_CONVERTIBLE\_PROPERTY~~ FIND\_REQUIRABLE\_PROPERTY (Property, SupportableProperties) ~~++is\_requirable~~ is well-formed ~~and has the value true~~.

*Returns:* A polymorphic wrapper whose target is the result of `std::require(e, p)`, where `e` is the target object of `*this`.

A similar change also needs to be made to `any_executor`'s `prefer` function. Combining this change with the one from § 5.3 Non-movable properties and changing the function from a non-member to a member as described in § 5.4 Polymorphic executor wrappers and prefer-only properties, the specification becomes:

```
template <class Property, class... SupportableProperties>
any_executor prefer(const any_executor<SupportableProperties...>& e, con
```

Let FIND\_PREFERABLE\_PROPERTY(p, \_pn) be the first type P in the parameter pack pn for which

- is\_same\_v<p, P> is true or is\_convertible\_v<p, P> is true, and
- P::is\_preferable is true.

If no such P exists, the operation FIND\_PREFERABLE\_PROPERTY(p, \_pn) is ill-formed.

*Remarks:* This function shall not participate in overload resolution unless

~~FIND\_CONVERTIBLE\_PROPERTY~~ FIND\_PREFERABLE\_PROPERTY (Property, SupportableProperties) ~~++is\_preferable~~ is well-formed ~~and has the value true~~.

*Returns:* A polymorphic wrapper whose target is the result of `std::prefer(e, p)`, where `e` is the target object of `*this`.

## § 5.6. `any_executor<P...>::target` causes unused RTTI overhead

The member functions `target` and `target_type` of `any_executor` cause the generation of RTTI for every executor type that is potentially wrapped in an `any_executor`. This RTTI overhead is present even if `target` and `target_type` are never called because `target_type` is a non-template member function. Some users will likely object to the RTTI overhead as going against C++ principle of "What you don't use, you don't pay for."

Using a property rather than a member function to retrieve the target executor from an `any_executor` reduces the RTTI to situations where the target is actually retrieved. The property can be user-defined and doesn't have to be part of the `any_executor` specification:

```
struct target_property_t
{
    using polymorphic_query_result_type = std::any;

    static constexpr bool is_requirable = false;
    static constexpr bool is_preferable = false;

    template<typename Ex>
    static constexpr bool is_applicable_property_v = std::execution::execution::is_applicable_property_v<Ex>;
};

template<typename Executor>
struct is_polymorphic_executor : std::false_type {};
template<typename ...Ps>
struct is_polymorphic_executor<std::execution::any_executor<Ps...>> : std::true_type {};
template<typename Executor>
inline constexpr bool is_polymorphic_executor_v = is_polymorphic_executor<Executor>::value;

template<typename Executor>
std::any query(Executor ex, target_property_t)
{
    static_assert(!is_polymorphic_executor_v<Executor>,
        "tried to query the target of a polymorphic executor which does not have a target");
    return std::any(ex);
}
```

(This proof of concept, with some example code of how to use it, can be found [here](#).)

If the committee is not overly concerned about the RTTI overhead, the specification can be left unchanged. If the committee likes this approach of using a property to retrieve the target executor, then the `target` and `target_info` members of `any_executor` should be removed and the target property can either be made a standard property or left to the user to define. The review group is not making any particular recommendation on this matter.

## § 5.7. `bulk_guarantee` specification should match execution policies

The specification `bulk_guarantee_t` in section 2.2.12.5 lists three possible values:

- `bulk_guarantee.unsequenced`: Execution agents within the same bulk execution may be parallelized and vectorized.
- `bulk_guarantee.sequenced`: Execution agents within the same bulk execution may not be parallelized.
- `bulk_guarantee.parallel`: Execution agents within the same bulk execution may be parallelized.

These correspond in some ways to the `execution_policies` for parallel algorithms, except that there are four execution policies, and `bulk_guarantee.unsequenced` seems to correspond to `parallel_unsequenced_policy` rather than `unsequenced_policy`.

The relationship between the `bulk_guarantee` property and the execution policies needs to be better defined. Any differences between them needs to be clearly explained. (At least in P0443, if not in the text of the standard.) The review group believes this is an issue that the authors of P0443 need to address. They are better equipped to come up with the right wording than either the review group or LEWG.

Daisy Hollman thinks it could be helpful to move the specification of `bulk_guarantee` to a separate paper, because a proper treatment is a paper-length topic and any hand-waving could be harmful to eventual success.

## § 5.8. What does "established property" mean?

Many places in the paper use the phrase "property already established" or something similar. But there is no precise definition of what "established" means. The paper should provide a definition, so that implementations (and users who write their own executor types) have a better understanding of what to implement.

Two obvious possible definitions of an established property might be:

1. `std::query(ex, prop) == prop_value` is true.
2. `std::require(ex, prop) == ex` is true (i.e. requiring the property has no effect).

Both of those definitions have flaws, and don't work correctly in all situations. We don't have a good suggestion of what the definition of an established property should be. Like the previous issue, we think this is something that the P0443 authors need to address.

(We have heard that the `static_thread_pool` review group is also suggesting how to define or reword "established property", at least for its uses in `static_thread_pool`. There may be overlap in this area.)

## § 5.9. `allocator_t::value` should not be static

Section 2.2.13.1 "`allocator_t` members" states:

```
static constexpr ProtoAllocator value() const;
```

*Returns:* The exposition-only member `a_`.

`value` is `static`, but the exposition-only member `a_` is not. A static member function cannot access a non-static data member.

We think the correct fix is to change `value` to be a non-static member function. One purpose of the `allocator` property is to get an executor object to use a particular allocator object. The `allocator` property needs to store that particular allocator object, so there is not always a generic value that can be returned by a static `value` function.

## § 5.10. `context` property has no constraints

The `context` property is a query-only property that can be used to get the execution context for an executor. For example, querying the `context` of an executor that came from a `static_thread_pool` will return the `static_thread_pool` object that created the executor.

The issue is that there are no requirements on the type or the value of the result of querying the `context` property. `std::query(ex, std::execution::context)` can be any type and doesn't have to satisfy any particular concept. This makes the `context` property difficult to use in generic

code. It is only really useful in code that knows something about the type of executor that it is working with.

We are not proposing that P0443 be changed to deal with this, since we didn't think of any way to improve the situation. We are just raising this as an issue to be aware of.

## § References

### § Informative References

#### [Asio]

Chris Kohlhoff. [Asio C++ library](https://github.com/chriskohlhoff/asio/tree/asio-1-17-0). URL: <https://github.com/chriskohlhoff/asio/tree/asio-1-17-0>

#### [P0443R13]

[A Unified Executors Proposal for C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0443r13.html). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0443r13.html>

#### [P1393]

[A General Property Customization Mechanism](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1393r0.html). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1393r0.html>

#### [P2033]

Jared Hoberock. [History of Executor Properties](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2033r0.pdf). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2033r0.pdf>